02-research-dimensions

02 — Research dimensions extraction

> Exhaustive extraction of the 12-dimension Boba browser-extension research corpus plus the insight and cross-verification documents. Source dir (paths contain spaces): `/Volumes/T7/Projects/Boba/docs/research/browser_extension/Browser Torrent Extension Guide/research`. This document is the requirements backbone for the implementation plan.

Files read

#	File	Lines	Topic
1	`boba_extension_dimensions.md`	49	Index of the 12 dimensions
2	`boba_extension_dim01.md`	990	Boba project architecture deep dive
3	`boba_extension_dim02.md`	2654	qBittorrent WebUI API v2 complete reference
4	`boba_extension_dim03.md`	2231	Browser extension architecture (MV3)
5	`boba_extension_dim04.md`	1667	Cross-browser

#	File	Lines	Topic
			compatibility matrix
6	`boba_extension_dim05.md`	1498	Tab Groups API (Chrome/ Yandex)
7	`boba_extension_dim06.md`	1604	Magnet link detection & parsing
8	`boba_extension_dim07.md`	2248	.torrent detection, download & bencode parsing
9	`boba_extension_dim08.md`	2076	Web page DOM scraping & dynamic content
10	`boba_extension_dim09.md`	3282	Extension ↔ Boba API integration
11	`boba_extension_dim10.md`	3908	Extension UI/ UX design patterns
12	`boba_extension_dim11.md`	1989	Security model & privacy architecture
13	`boba_extension_dim12.md`	3364	Testing, build system & store distribution
14	`boba_extension_insight.md`	129	Cross-dimension insights (10 insights)
15	`boba_extension_cross_verification.md`	90	Cross-verification (tiers,

#	File	Lines	Topic
			conflicts, conclusions)
	TOTAL	**27,779**	(corpus header states 27,560 research lines across 12 dims)

Dimension 01 — Boba Project Architecture Deep Dive

Covers (Dim01 §1–15): Complete architecture of `milos85vasic/Boba-Base` — a multi-tracker meta-search platform for qBittorrent: service topology, full REST API catalog (Python FastAPI + Go/Gin), request/response schemas, auth mechanisms, data models, the qBittorrent nova3 plugin system, search/download flows, env vars, Angular 21 frontend, hook system, and the extension integration points.

Service topology / ports (Dim01 §1, table): - qBittorrent WebUI — port **7185** (container `qbittorrent`, C++ upstream). [This is the authoritative repo port; cross_verification #9 notes qBittorrent "typically on 8080" generically — Dim09 uses 8080, the conflict is C4-adjacent.] - Merge Search (FastAPI, Python 3.12) — port **7187** (container `qbittorrent-proxy`); main API, search orchestration, serves the Angular SPA. - Download Proxy (Python) — port **7186** (same container; legacy passthrough). - WebUI Bridge (`webui-bridge.py`, host process, Python 3) — port **7188**; private-tracker auth, theme injection. - Jackett (C#) — port **9117**. - boba-jackett (Go/Gin) — port **7189**; Jackett management API. - Go Proxy (`qbittorrent-proxy-go`, Gin) — port **7187** under compose profile `go` (alternative merge backend). - Optional quality stack (`docker-compose.quality.yml`): SonarQube :9000, Prometheus :9090, Grafana :3000.

FastAPI primary API (port 7187) — exact endpoints (Dim01 §2.1): - Search: `POST /api/v1/search` (async fire-and-forget → SearchResponse status running), `POST /api/v1/search/sync` (blocking legacy), `GET /api/v1/search/{search_id}`, `GET /api/v1/search/stream/{search_id}` (SSE text/event-stream), `POST /api/v1/search/{search_id}/abort`. - Download: `POST /api/v1/download` (add torrent(s) to qBittorrent →

{download_id, status, added_count, results}), POST /api/v1/download/file (returns Blob application/x-bittorrent), POST /api/v1/magnet ({result_id, download_urls} → {magnet, hashes}), GET /api/v1/downloads/active. - Auth: POST /api/v1/auth/qbittorrent ({username,password,save?}), GET /api/v1/auth/rutracker/status, GET /api/v1/auth/rutracker/captcha, POST /api/v1/auth/rutracker/login, POST /api/v1/auth/rutracker/cookie-login, GET /api/v1/auth/status (all trackers), POST /api/v1/auth/qbittorrent/logout. - Hooks: GET/POST /api/v1/hooks, DELETE /api/v1/hooks/{hook_id}, GET /api/v1/hooks/logs?limit=50&hook_name=. - Schedules: GET/POST /api/v1/schedules, GET/PATCH/DELETE /api/v1/schedules/{id}. - Theme: GET/PUT /api/v1/theme, GET /api/v1/theme/stream (SSE). - Utility: GET /health ({status, service, version}), GET /api/v1/bridge/health, GET /api/v1/config (returns qbittorrent_url etc.), GET /api/v1/stats, GET /docs (Swagger), GET /openapi.json (OpenAPI 3.1, frozen for CI diffing).

Go boba-jackett API (port 7189) (Dim01 §2.2): GET /healthz, GET /openapi.json, GET/POST /api/v1/jackett/credentials, GET /api/v1/jackett/indexers, GET /api/v1/jackett/catalog, POST /api/v1/jackett/catalog/refresh, GET /api/v1/jackett/autoconfig/runs, POST /api/v1/jackett/autoconfig/run, GET/POST /api/v1/jackett/overrides. Go backend differences: uses qBittorrent native search API (/api/v2/search/start) instead of plugin subprocesses; SQLite (boba.db at /config/boba.db); admin/admin Basic Auth for mutating endpoints.

qBittorrent WebUI native (port 7185, proxied via 7186/7188) (Dim01 §2.3): POST /api/v2/auth/login (returns "Ok."), GET /api/v2/app/version, POST /api/v2/torrents/add, GET /api/v2/torrents/info, POST /api/v2/search/start, GET /api/v2/search/results, POST /api/v2/search/stop, GET /api/v2/search/plugins.

Schemas (Dim01 §3) — exact defaults: - SearchRequest: query (required, min_length=1), category (default 'all'), limit (1–100, default **50**), enable_metadata (default true), validate_trackers (default true), sort_by (default 'seeds'), sort_order (asc|desc, default desc). - SearchResponse: search_id (uuid), status enum running|completed|failed|no_results|captcha_required, results[], total_results, merged_results, trackers_searched[], errors[], tracker_stats[], started_at/completed_at (ISO-8601). - SearchResultResponse: name, size, seeds, leechers, download_urls[], quality (uhd_4k|full_hd|hd|sd|unknown), content_type (movie|tv|anime|music|game|software|ebook|other), desc_link, tracker, sources[], metadata{source(OMDb|TMDB), title, year, poster_url, overview, genres[]}, freeleech (default false). - DownloadRequest: {result_id, download_urls[]}. - TrackerSearchStat: name, tracker_url, status (pending|running|success|empty|error|

`timeout|cancelled`), results_count, started_at/completed_at, duration_ms, error, error_type (`upstream_http_403|dns_failure|plugin_crashed|null`), authenticated, attempt, http_status, category, query, notes.

Auth mechanisms (Dim01 §4): qBittorrent default creds **admin/admin** (cookie-based, `SID` cookie; saved creds at `/config/download-proxy/qbittorrent_creds.json`). Private trackers: RuTracker (env `RUTRACKER_USERNAME/PASSWORD` + CAPTCHA flow + cookie-login alternative), Kinozal (env, falls back to IPTorrents creds), NNM-Club (cookie-only `NNMCLUB_COOKIES`, proxy-sensitive), IPTorrents (env, **freeleech-only enforced**). Jackett: `JACKETT_API_KEY`. boba-jackett: admin/admin Basic; GET/HEAD/OPTIONS pass through unauthenticated, mutating requires `Authorization: Basic YWRtaW46YWRtaW4=`. **CORS**: `ALLOWED_ORIGINS` env (default `*` in dev).

Data architecture (Dim01 §5, §15): No relational DB — all state in-memory dataclasses + JSON files (`hooks.json`, `qbittorrent_creds.json`, `scheduling.json`, `theme.json`); SQLite `boba.db` only for the Go backend. Dataclasses: SearchMetadata, SearchResult, MergedResult, CanonicalIdentity (dedup fingerprint: infohash/title/year/content_type/season/episode/resolution/codec/group/metadata_source). ContentType enum: movie/tv/anime/music/audiobook/game/software/ebook/other/unknown. QualityTier enum: sd/hd/full_hd/uhd_4k/uhd_8k/unknown.

Plugins (Dim01 §6): 48 built-in (qBittorrent nova3 format, installed to `/config/qBittorrent/nova3/engines/`); plugin contract = class with `url`, `name`, `supported_categories`, `search(what,cat)`, `download_torrent(url)`. Public trackers fan out via subprocess; private via aiohttp + session cookies. Dedup tiers: infohash > normalized title+size > Levenshtein fuzzy > weak heuristic. **IPTorrents freeleech results never merged with non-freeleech** (preserves ratio). Dead trackers (excluded unless `ENABLE_DEAD_TRACKERS=1`): ali213, audiobookbay, bitru, bt4g, btsow, extratorrent, eztv, one337x, pctorrent, solidtorrents, therarbg, torrentfunk, xfsub, yihua. PRIVATE_TRACKERS: rutracker (rutracker.org), kinozal (kinozal.tv), nnmclub (nnm-club.me), iptorrents (iptorrents.com).

SSE event types (Dim01 §7.2): `search_start`, `tracker_started`, `tracker_completed`, `result_found`, `results_update`, `search_complete`, `error`, `close`.

Performance env defaults (Dim01 §9.3): MAX_CONCURRENT_SEARCHES=5, MAX_CONCURRENT_TRACKERS=10, MAX_CONCURRENT_SSE_STREAMS=32,

PUBLIC_TRACKER_DEADLINE_SECONDS=15, PLUGIN_TIMEOUT=10, SSE_TIMEOUT=30. CAPTCHA: PENDING_CAPTCHAS_MAX=1024, PENDING_CAPTCHAS_TTL_SECONDS=900.

Frontend (Dim01 §10): Angular 21 SPA, `HttpClient` with `baseUrl = ''` (same-origin, served from :7187), `EventSource` for SSE. Hook events (Dim01 §11): search_start, search_progress, search_complete, download_start, download_progress, download_complete, merge_complete, validation_complete.

Extension integration strategy (Dim01 §12): Extension talks directly to FastAPI :7187 — search via `/api/v1/search` + SSE stream, download via `/api/v1/download`, get torrent file via `/api/v1/download/file`, generate magnet via `/api/v1/magnet`, check auth via `/api/v1/auth/status`, get config via `/api/v1/config`. Production CORS: `ALLOWED_ORIGINS=http://localhost:7187,chrome-extension://YOUR_EXTENSION_ID`. Extension background scripts not subject to CORS. Recommended architecture: background SW (SSE, cache, downloads, creds) + popup (React/Vue) + content scripts + options page.

Cross-refs: Dim02 (qBittorrent API), Dim09 (integration), Dim11 (security — hardcoded admin/admin noted as a risk).

Dimension 02 — qBittorrent WebUI API v2 Complete Reference

Covers (Dim02 §1–20): Every WebUI API v2 namespace/endpoint/parameter/response for v4.1+ through v5.2+, auth (cookie + API key), add/upload, monitoring, management, preferences, categories/tags, search, RSS, log, error handling, v4↔v5 version diffs, the Python `qbittorrent-api` library, and SSE/WebSocket reality.

Base/general (Dim02 §1): `/api/v2/<namespace>/<method>`. GET for reads, POST for mutations/uploads. **v4.4.4+ returns 405** on wrong method. All methods require auth **except** `/api/v2/auth/login`. Namespaces: auth, app, torrents, sync, transfer, log, rss, search.

Authentication (Dim02 §2): - Cookie-based (all versions): `POST /api/v2/auth/login` form `username&password` → 200 sets `SID` cookie / **403 = IP banned** (too many failures). **CRITICAL CSRF:** `Referer` (or `Origin`) header MUST equal the Host exactly (a `null` Origin is also accepted). Use `credentials:'include'`. Logout: `POST /api/v2/auth/logout`. - API key auth (v5.2.0+, API v2.14.1+): 32-char key, prefix `qbt_` + 28 alnum, 160 bits entropy, **only one key at a time**, rotation invalidates prior. Header `Authorization: Bearer qbt_...`. Cannot fetch WebUI/static assets, cannot use auth endpoints.

Add torrents `/api/v2/torrents/add` (Dim02 §3–5): POST multipart. Params: `urls` (newline-separated; http/https/magnet/bc://bt), `torrents` (raw bytes, repeatable for batch), `savepath`, `cookie` (removed in API v2.11.3 — use `app/cookies`), `category`, `tags` (comma-sep), `skip_checking`, `paused`, `root_folder`, `rename`, `upLimit`, `dlLimit`, `ratioLimit` (v2.8.1+), `seedingTimeLimit` (min, v2.8.1+), `autoTMM`, `sequentialDownload`, `firstLastPiecePrio`, `content_layout` (`Original|Subfolder|NoSubfolder`, supersedes root_folder since v2.7), `downloadPath` (v5.0), `stopCondition` (`None|MetadataReceived|FilesChecked`, v5.0), ssl_certificate/ ssl_private_key/ssl_dh_params (v5.0, API v2.10.4), `is_stopped` (v2.11.0), `forced` (v2.11.0). **Returns 415 = invalid torrent; 200 = success (200 does NOT guarantee added).**

Monitoring (Dim02 §6): `/torrents/info` (filter values: all/downloading/ seeding/completed/paused/active/inactive/resumed/stalled/ stalled_uploading/stalled_downloading/errored; v5.0+ replaces `paused` with `stopped`, adds `running`; supports category/tag/sort/reverse/limit/ offset/hashes). Full torrent fields documented (hash, state, progress, dlspeed, eta, ratio, num_seeds, `isPrivate/infohash_v1/infohash_v2` v5.0+, magnet_uri, etc.). **Torrent states** (key v5 rename: `pausedUP→stoppedUP`, `pausedDL→stoppedDL`). `/sync/maindata?rid=` delta sync (rid=0 = full; maintain local state + merge). `/torrents/properties`, `/trackers`, `/files` (priority 0=skip,1=normal,6=high,7=maximal).

Management (Dim02 §7): stop (v5)/pause (v4); start (v5)/resume (v4); delete (`hashes+deleteFiles` default false since v2.7); recheck; reannounce; setCategory; setTags (v5.1)/addTags+removeTags; setAutoManagement; toggleSequentialDownload; setForceStart; setDownloadLimit/setUploadLimit (bytes/s, -1/0 unlimited); setShareLimits (ratioLimit -2 global/-1 unlimited; seedingTimeLimit minutes -2/-1); setLocation; rename; priority (increasePrio/ decreasePrio/topPrio/bottomPrio — **409 if queueing disabled**); filePrio. Hashes are `|`-separated or `all`.

Preferences/categories/tags/search/rss/log (Dim02 §8–13): `app/preferences` & `setPreferences` (json string, strings quoted, ints/bools not). `app/version`, `app/webapiVersion`, `app/buildInfo`, `app/defaultSavePath`. Categories: getAll/createCategory(savePath optional)/ editCategory/removeCategories(newline-sep). Tags: tags/ createTags(comma)/deleteTags/addTags/removeTags. **Search API:**

`search/start` (`pattern`+`plugins`(`|`/all/enabled)+`category`; **409 if max 5 concurrent**), search/stop, search/status (Running/Stopped), search/results (id,limit,offset), search/delete, plugins/installPlugin/uninstallPlugin/enablePlugin/updatePlugins. RSS (experimental). Log (`log/main` types 1=Normal/2=Info/4=Warning/8=Critical; **timestamps were ms before v4.5.0, seconds since**).

Error codes (Dim02 §14): 200 OK, 400 bad request, 401 XSS/host-header fail, 403 not logged in/banned, 404 not found, **405 wrong method (v4.4.4+)**, 409 conflict (queueing not enabled / category missing), 415 invalid torrent, 500 server error. **No native WebSocket/SSE** — all real-time via polling; recommend `/sync/maindata` 1–5s (one user: ~60s with ~4000 torrents on Atom CPU); for "add torrent" no polling needed.

Version diffs (Dim02 §15): New in v5.0+: isPrivate, downloadPath, stopCondition, is_stopped (replaces paused), forced; setTags (v5.1); app/cookies+setCookies (v2.11.3); API key auth (v5.2.0, API v2.14.1). Python lib `qbittorrent-api` auto-refreshes cookies.

Extension notes (Dim02 §19): Store creds in `chrome.storage.local` (not sync) and consider encrypting; Referer must match host; implement exponential backoff to avoid IP bans; HTTPS for remote.

Cross-refs: Dim01, Dim09 (both reference cookie auth + add endpoint — cross_verification #1, #2).

Dimension 03 — Browser Extension Architecture (Manifest V3)

Covers (Dim03 §1–17): MV3 manifest structure, service worker lifecycle, content scripts, fetch, messaging, storage, action, notifications, permissions, offscreen documents, CORS, packaging, a full reference implementation, and Firefox differences.

Manifest (Dim03 §1): Only `manifest_version`, `version`, `name` are mandatory. `host_permissions` is a separate key from `permissions` in MV3. SemVer `major.minor.patch.build`. Recommended torrent-ext permissions: storage, notifications, contextMenus, scripting, alarms, activeTab; optional clipboardWrite; host `http://*/` `https://*/`; optional_host `<all_urls>`. Background: `{service_worker, type:"module"}`.

Service worker lifecycle (Dim03 §2) — HARD timeouts: terminate after **30s of inactivity**, OR a single request >**5 min**, OR a `fetch()` response >**30s**. Chrome version behaviors: Chrome 120 alarms min 30s period; 116 WebSocket extends lifecycle; 114 persistent messaging keeps SW alive; 110/109 API calls/offscreen messages reset idle timer. Keep-alive: **chrome.alarms** (official; wakes SW even after termination) — `periodInMinutes: 0.5` (30s, Chrome 120+); long-lived ports; `waitUntil()` helper (25s interval `getPlatformInfo`). **CRITICAL RULE:** all event listeners MUST be registered synchronously at top level — listeners inside async callbacks don't survive SW restarts.

Content scripts (Dim03 §3): `run_at` = document_start (loading) / document_end (interactive) / **document_idle (default, complete — recommended for torrents** since magnets load dynamically). Match patterns <scheme>://<host><path>; <all_urls> matches all permitted schemes. Worlds: ISOLATED (default; separate JS scope, shares DOM, has chrome.runtime) vs MAIN (shared page scope, NO chrome.runtime — page can detect/interfere). Regexes: `MAGNET_REGEX = /magnet:\?xt=urn:[a-z0-9]+:[a-z0-9]{32,40}/gi`, `TORRENT_FILE_REGEX = /\.torrent($|\?|&)/i`, `TRACKER_REGEX = /(udp|http|https):\/\/[^\s"'<>]+\/(announce|scrape)/gi`.

Messaging (Dim03 §5): `chrome.runtime.sendMessage` / `tabs.sendMessage` / `runtime.connect` (ports). **`return true` from onMessage is REQUIRED for async sendResponse.** Long-lived ports also keep SW alive.

Storage (Dim03 §6): local ~**10 MB** (unlimitedStorage to raise), sync ~**100 KB total / 8 KB per item** (cross-device), session ~**10 MB** (in-memory, lost on browser close — for SW runtime state), managed (admin read-only). `storage.onChanged` for cross-context sync.

action/notifications/permissions (Dim03 §7–9): Badge text recommended **≤4 chars**; `setBadgeText`/`setBadgeBackgroundColor` (per-tab via tabId). Context menus created in `runtime.onInstalled`, handled via `contextMenus.onClicked`. Notifications: 4 templates (basic/list/progress/image); **MV3 no Base64 data-URL icons — file path required**; max 2 buttons; events onClicked/onButtonClicked/onClosed. Permissions categories: permissions (install), host_permissions (install; Firefox may need manual enable), optional_permissions/optional_host_permissions (runtime, **must be requested synchronously in a user-gesture handler**). `activeTab` grants temp access on user gesture + auto-grants scripting on that tab. `declarativeContent.PageStateMatcher` shows action only when CSS conditions met without persistent host permissions.

Offscreen documents (Dim03 §10): Provide DOM in MV3 (DOMParser, clipboard, blobs, etc.); one per profile; reasons include DOM_PARSER, DOM_SCRAPING, CLIPBOARD, BLOBS, etc. Use for parsing HTML / .torrent files.

CORS (Dim03 §11): Extensions bypass CORS for `host_permissions` hosts via SW fetch (target server must still send appropriate CORS headers). Content scripts CANNOT fetch cross-origin — route through SW. Chrome adds `Origin: chrome-extension://<id>` header (cannot be overridden).

Packaging (Dim03 §12): `.crx` = signed ZIP; first pack generates `.pem` (keep secure — extension ID = hash of public key). Self-hosted update: `update.xml` (Omaha format: appid/codebase/version) + manifest `update_url`.

Firefox diffs (Dim03 §15): Firefox MV3 uses **Event Pages (background scripts), NOT service workers**; supports BOTH MV2 + MV3 (no deprecation); blocking webRequest still works in MV3; offscreen documents NOT supported; needs `service_worker` field (Firefox 121+) or `browser_specific_settings.gecko.id` + strict_min_version.

Gotchas (Dim03 §17): SW dies after 30s (use alarms not setInterval); top-level listeners; `return true`; no DOM in SW; no Base64 notification icons; server must accept `chrome-extension://` origin; permission request in click handler; storage.session not persistent; badge ≤4 chars.

Cross-refs: Dim04 (cross-browser), Dim11 (isolated world = security).

Dimension 04 — Cross-Browser Compatibility Matrix

Covers (Dim04 §1–12 + appendices): Chrome/Firefox/Opera/Yandex/Chromium/Edge compatibility, manifest differences, `chrome.*` vs `browser.*`, webextension-polyfill, runtime detection, build pipeline, store submission requirements.

Executive findings (Dim04 §1): Chrome **mandates MV3 (MV2 removed from CWS June 2025)**; Firefox supports MV2+MV3 (no deprecation, blocking webRequest works in MV3); Opera/Yandex/Edge are Chromium (`chrome.*`); webextension-polyfill bridges Promise `browser.*` on Chromium; WXT/vite-plugin-web-extension automate multi-browser manifests.

Engine/API/store per browser (Dim04 §2.1): Chrome (Blink, chrome., *MV3 only, CWS), Firefox (Gecko, browser.*+chrome.*, MV2+MV3, AMO), Opera (Blink, chrome.*, Opera Add-ons), Yandex (Blink, chrome.*, MV3, **CWS or `browser://tune/`**), Chromium (Blink, manual install), Edge (Blink v79+, chrome.*, Edge Add-ons). UA detection (order matters): Yandex `YaBrowser`/`Yowser`, Opera `OPR/`, Edge `Edg/`, Chrome `Chrome`+not-Opera/Edge, Chromium `Chromium`. **Use feature detection primarily; UA as last resort.**

Compatibility matrix (Dim04 §3.1): Service worker required on all Chromium MV3; Firefox uses event pages. `browser.webRequest` blocking only Firefox; Chromium uses declarativeNetRequest (DNR). `storage.sync`/`storage.managed` **NOT supported on Opera**. Offscreen docs not in Firefox. `sidebarAction` only Firefox+Opera. `identity` Opera only `launchWebAuthFlow`. Firefox `browser.menus` is a superset of `contextMenus` (polyfill maps them).

Manifest diffs (Dim04 §4): Firefox MV3 background uses `scripts` (not `service_worker`) and **REQUIRES `browser_specific_settings.gecko.id`** (AMO does NOT auto-assign for MV3) + `strict_min_version` (e.g. 109.0). Opera: `minimum_opera_version`, `sidebar_action`. Per-browser template via `{{chrome}}.` `{{firefox}}.` prefixes.

Polyfill (Dim04 §6): webextension-polyfill — Chrome officially supported, Firefox NO-OP, Opera/Edge (≥79.0.309) unofficially supported. **Does NOT polyfill Firefox-only APIs onto Chrome** — extension must do its own runtime feature detection. Install `webextension-polyfill` + `@types/webextension-polyfill`. Limitations: no callback support, tabs.executeScript Chrome only immediate values, missing-API not polyfilled, depends on api-metadata.json.

Build pipeline (Dim04 §8): Recommended **WXT** (Vite-based, supports all browsers, MV2+MV3, HMR, per-browser manifest generation, file-based entrypoints). `wxt build --browser chrome|firefox|edge|opera`. Output `.output/chrome-mv3/`, `firefox-mv2/`, etc. Alternatives: vite-plugin-web-extension, manual scripts.

Store requirements (Dim04 §9) — concrete: - **Chrome Web Store:** Google account + **$5 one-time fee** (covers all extensions, up to 20/account, valid for life); MV3 required; 128×128 PNG icon (96×96 actual + 16px padding); screenshots 1280×800 or 640×400 (min 1, max 5); small promo tile 440×280; large promo 1400×560 (optional); description ≤16,000 chars; privacy policy if data collected; single purpose; permission justification; remote code = "No"; review automatic <1h; **2FA required**. - **Firefox AMO:** Free; MV2/MV3; **extension ID required for MV3**; icon 48/96/128; screenshots 1280×800 or 640×480; **data_collection_permissions required for new extensions since Nov 3 2025** (`browser_specific_settings.gecko.data_collection_permissions`); source code may be required if minified; auto + manual review; unlisted self-distribution available; **version format `x.y.z` only (no letters)**. - **Opera Add-ons:** Free; MV2/MV3; icons 128/48/16; screenshots **612×408 preferred (max 800×600), white background, other extensions disabled**; no external JS; **manual review, no SLA**; no 2FA. - **Edge Add-ons:** Free; MV2/MV3; icon 300×300 (min 128, 1:1); screenshots 640×480 or 1280×800 (max 6); description 250–10,000 chars; **2FA required**; manual review no SLA. - **Yandex:** No own store — install via CWS, `browser://tune/` (drag CRX3), or unpacked at `browser://extensions/`; checks against malicious-extension DB; ignores extensions that change new-tab. - **Chromium:** No store; developer mode / `--load-extension` (removed from branded Chrome v137+ — use `--remote-debugging-pipe` + `Extensions.loadUnpacked`) / drag CRX.

Permission mapping (Dim04 A.3): webRequest = DNR-only on Chromium, blocking on Firefox. Notifications "Partial" on Opera (no Image/List/Progress on Mac).

Cross-refs: Dim03 (MV3), Dim05 (Yandex tab groups), Dim12 (build/distribution).

Dimension 05 — Tab Groups API (Chrome/Yandex)

Covers (Dim05 §1–16): `chrome.tabGroups` reference, `chrome.tabs` integration, permissions, TabGroup type, group→URL enumeration, events, SW processing, context menus, collapsed/cross-window behavior, errors, Yandex specifics, edge cases.

Availability (Dim05 §2): `chrome.tabGroups` requires **Chrome 89+, MV3+** (undefined earlier). `TAB_GROUP_ID_NONE === -1`. Methods (Chrome 90+): `get(groupId)`, `query(queryInfo{collapsed?,color?,shared?(Chrome137+),title?(partial),windowId?})`, `update(groupId,{collapsed?,color?,title?})`, `move(groupId,{index,windowId?})`. **`tabGroups` API CANNOT create/alter/remove groups** — use `tabs.group()/tabs.ungroup()` (Chrome 88+).

tabs integration (Dim05 §3): `tabs.group({tabIds, groupId?, createProperties{windowId?}})` → groupId (validation error → tabs NOT modified). `tabs.ungroup(tabIds)` (empty group auto-deleted). `tabs.query({groupId})` (`TAB_GROUP_ID_NONE` for ungrouped). `Tab.groupId` property.

Permissions (Dim05 §4): `tabGroups` perm → warning “View and manage your tab groups” (**not shown as alarming / quiet permission per MDN**); accessing `tab.url/title/favIconUrl` **requires `tabs` permission** (or host permissions) else undefined. **tabs.group/ungroup and tabs.query({groupId}) do NOT require tabGroups permission** (only `tabGroups.*` methods do).

TabGroup type (Dim05 §5): Color enum = grey/blue/red/yellow/green/pink/purple/cyan/orange. Fields: id (**session-unique, NOT persistent across restarts — identify by title+color+member URLs**), title, color, collapsed, windowId, shared (Chrome 137+).

Enumeration (Dim05 §6): `tabGroups.query({})` (all windows) → for each `tabs.query({groupId:group.id})` → map URLs. The Boba “send group to Boba” workflow extracts every tab URL.

Events (Dim05 §7): onCreated/onMoved/onRemoved/onUpdated (all `(group)`). **onMoved does NOT fire on cross-window move** (fires onRemoved+onCreated, NEW groupId). Tab add/remove from group detected via `tabs.onUpdated` `changeInfo.groupId`.

Collapsed/cross-window (Dim05 §10–11): Collapsing hides tabs in UI but **does NOT restrict API access** to tabs/URLs. Groups cannot span windows; move only between `windows.WindowType==="normal"`.

Errors (Dim05 §12): “No group with id” (deleted), invalid tab id, **Chrome 148+ chrome://newtab cannot be grouped** (“Grouping is not supported by tabs in this window” — use about:blank), Saved Tab Groups cannot be `update()`d while not open (Chromium bug 323982812).

Yandex (Dim05 §13): Chromium-based, currently **~Chrome 147+** (Yandex 26.3, April 2026) — well above the Chrome 89 minimum; full tabGroups support; native tab groups since ~2021; own sync; own extension store; checks against malicious-extension DB.

Edge cases (Dim05 §15): groupId reuse across sessions; incognito groups don't persist; **pinned tabs auto-unpinned before grouping**; max ~32–64 groups/window; chrome:// restricted tabs may not group; Brave `update()` title may not render until user interacts (bug 52949); ungroup last tab deletes group; **Firefox uses a different (incompatible) tab groups API — this feature is Chrome/Yandex only**.

Cross-refs: Dim01 (batch torrent flow), Dim04 (Yandex), Dim10 (UI/UX).

Dimension 06 — Magnet Link Detection & Parsing Algorithms

Covers (Dim06 §1–15 + appendices): BEP 9 magnet URI spec, BTIH v1/v2 formats, detection regexes, URI decoding, xt/dn/tr handling, JS/TS parser, validation, edge cases, generation, base32↔hex conversion, library comparison, performance, security.

Spec (Dim06 §1, BEP 9): magnet:?xt=urn:btih:<hash>&dn=<name>&tr=<tracker>&x.pe=<peer>. **`xt` is the ONLY mandatory parameter.** Params: xt (eXact Topic, required), dn (display name), tr (tracker, repeatable), xl (length bytes), xs (exact source), as (acceptable source), ws (web seed, BEP 19), kt (keyword), mt (manifest), so (select-only, BEP 53), x.pe (peer), x.* (experimental). Multiple files: repeat `xt=` OR numbered `xt.1=`/`xt.2=`.

Hash formats (Dim06 §2): v1 hex = `urn:btih:` + **40 hex** (SHA-1, 20 bytes); v1 base32 = `urn:btih:` + **32 base32 chars** (RFC 4648, legacy/Vuze — must convert to hex); v2 = `urn:btmh:1220` + **64 hex** (SHA-256 multihash, prefix `0x12 0x20`); hybrid = both prefixes. Example v1 hex `d2474e86c95b19b8bcfdb92bc12c9d44667cfa36`; base32 `QHQXPYWMACKDWKP47RRVIV7VOURXFE5Q`.

Detection regex (Dim06 §3): Two-phase recommended — Phase 1 broad candidate `/magnet:\?\S+/gi`, Phase 2 validate xt: `/^urn:bti[h]?:[a-fA-F0-9]{40}$/i` OR `/^urn:bti[h]?:[A-Z2-7]{32}$/i` OR /

`^urn:btmh:1220[a-fA-F0-9]{64}$/i`. Documented test cases (valid hex/base32/v2/hybrid/multiple-xt/numbered-xt/embedded; invalid: too short, missing xt, empty, unknown urn, extra chars after hash).

Decoding (Dim06 §4): `dn` → decodeURIComponent then +→space; `tr`/`xs`/`as`/`ws` → decodeURIComponent; `kt` → split on +; `ix`/`xl` → Number; `so` → parse BEP 53 ranges (`0,2,4-6` → [0,2,4,5,6]). Duplicate params → array. Trackers deduplicated via Set (only http/https/udp protocols valid).

Parser/validation (Dim06 §8–9): Full TS `decodeMagnetURI`/`encodeMagnetURI`, base32Decode/bytesToHex, extractInfoHashes (normalizes base32 to hex), `isValidInfoHashV1` (40 hex), `isValidInfoHashV2` (64 hex), `isValidMagnetURI`, `validateMagnetStrict`. Supports `stream-magnet:` scheme too.

Edge cases (Dim06 §10): only-xt, trackerless (DHT-only), base32, mixed-case (normalize lowercase), duplicate trackers, hybrid, multiple/numbered xt, empty/encoded/unicode dn, x.pe, BEP 53 so, BEP 46 btpk, truncated, invalid hex, wrong length, missing `magnet:?`, trailing text.

base32↔hex (Dim06 §12): qBittorrent auto-converts base32→hex on add. `base32ToHex('WRN7ZT6NKMA6SSXYKAFRUGDDIFJUNKI2')` → `b45bfccfcd5301e94af8500b1a1863415346a91a`.

Libraries (Dim06 §13): `magnet-uri` (WebTorrent, most-used; deps @thaunknown/thirty-two, bep53-range, uint8-util; no tracker URL validation; no stream-magnet); `parse-torrent` (broader, parses .torrent too, larger); `@ctrl/magnet-link` (TS port, fewer deps); **custom = smallest, 0 deps, TS types** — recommendation table favors custom or @ctrl for extensions.

Performance/security (Dim06 §14–15): DOM-first `querySelectorAll('a[href^="magnet:"]')` is orders of magnitude faster than text scanning; chunk large text (100KB) to avoid catastrophic backtracking; two-phase validation; deduplicate by infohash; debounce MutationObserver scans. **Sanitize `dn` before HTML render** (XSS): strip control chars + <>, limit 255 chars, use textContent. Strict CSP `script-src 'self'; object-src 'self'`.

Cross-refs: Dim07 (infohash from .torrent), Dim08 (DOM scanning), Dim09 (unified identity / dedup).

Dimension 07 — Torrent File Detection, Download & Bencode Parsing

Covers (Dim07 §1–15 + appendices): BEP 3/52 metainfo format, bencode spec, .torrent link detection, CORS-aware download, bencode parsing in JS/TS, infohash computation, single/multi-file structure, piece validation, magnet generation, private torrents (BEP 27), validation, blob/arraybuffer handling, complete impl, edge cases, library comparison.

Format (Dim07 §1, BEP 3): .torrent = bencoded dict with required `announce` (tracker URL) + `info` (dict). Optional top-level: announce-list (BEP 12 tiered), creation date, comment, created by, encoding. **info dict:** required piece length (power of 2: 256KB=2^18, 1MB=2^20, 4MB=2^22), pieces (concat of **20-byte SHA-1** hashes, length multiple of 20), name, private (1 ⇒ disable DHT/PEX). Single-file: `length`. Multi-file: `files[]` (each `{length, path[]}`). **v2 (BEP 52, Draft):** SHA-256 infohash, SHA-256 piece hashes, merkle tree per file, `urn:btmh:` prefix; for DHT/trackers the SHA-256 infohash truncated to 20 bytes.

Bencode (Dim07 §2): byte string `<len>:<content>` (binary-safe, `0:` valid); integer `i<num>e` (no leading zeros except `i0e`, `i-0e` invalid); list `l...e`; dict `d...e` (**keys must be byte strings in lexicographically-sorted RAW-byte order** — essential for deterministic infohash). Decode by first byte: i/l/d/digit/e.

.torrent detection (Dim07 §3): `/\.torrent(?:\?.*)?(?:#.*)?$/i`; download-endpoint + passkey patterns (`/\w{32}/filename.torrent`).

CORS download (Dim07 §4): Content scripts CANNOT cross-origin fetch since Chrome 85 — MUST delegate to SW (which fetches with `host_permissions`). MIME `application/x-bittorrent`. Read as ArrayBuffer → Uint8Array. `no-cors` returns opaque (only useful for HEAD existence check).

Bencode parsing libs (Dim07 §5, §15): `@ctrl/torrent-file` (native Uint8Array, no Buffer — **recommended for extensions**, ~8KB gzip, MV3-compatible, v2 support); `@substrate-system/bencode`; `bencode-js`; `parse-torrent/node-bencode` (need Buffer polyfill). Full zero-dep `BencodeDecoder`/`BencodeEncoder` provided (encoder sorts dict keys).

Infohash computation (Dim07 §6) — CRITICAL: Infohash = **SHA-1 of the RAW bencoded `info` dictionary as it appears in the file** (NOT decode-then-re-encode, which can reorder bytes). Per BEP 52, must reject invalid metainfo or extract substring directly — never decodeencode roundtrip on invalid data. Provided `extractInfoDict()` finds `4:info`, tracks nesting depth to find matching `e`, then `crypto.subtle.digest('SHA-1', infoDictBytes)` → hex. Web Crypto SHA-1 available Chrome 37+/Firefox 34+/Safari 7+/Edge 12+, **secure contexts (HTTPS) only**, works in SW/Web Workers.

Structure/pieces/magnet (Dim07 §7–9): Total size = `length` (single) or sum of `files[].length` (multi). Path = `[name, ...path]` joined `/`. Pieces split into 20-byte SHA-1 hashes; verify via constant-time compare. `generateMagnetUri({infoHash,name,trackers,size,webSeeds})` (URLSearchParams, `tr.append` for multiple) / `torrentToMagnet(data)`.

Private torrents (Dim07 §10, BEP 27): `info.private===1` ⇒ MUST disable DHT/PEX/LSD, tracker-only. Passkey in announce URL (`/UNIQUE_PASSKEY/announce`) is **sensitive — never log/share; sanitize to `***PASSKEY***`**.

Validation (Dim07 §11): require info+announce/announce-list; info.name/piece length/pieces (length %20==0); either length XOR files; valid piece lengths [32768…8388608]; per-file length+path; valid announce URL.

Blob handling (Dim07 §12): `response.arrayBuffer()`→Uint8Array; size limit (≤10MB typical, most <1MB; example max 50MB); validate first byte `0x64` ('d'). Constant-time equals helper.

Edge cases (Dim07 §14): empty file, invalid bencode (positioned errors), missing info, missing length+files, non-standard piece length (warning), unicode/binary filenames (TextDecoder utf-8 → fallback iso-8859-1), unterminated structures, leading zeros, negative zero, out-of-order keys (critical for infohash), CORS failure, timeout (AbortController), malformed magnet, robust retry (don't retry 404/parse errors). BEP reference: BEP 3 (Final), 9 (Final), 12, 27, 47, 52 (Draft), 53 (Draft).

Cross-refs: Dim06 (magnet/infohash, cross_verification #6,#7), Dim09 (download/integration).

Dimension 08 — Web Page DOM Scraping & Dynamic Content Detection

Covers (Dim08 §1–15): content-script timing, DOM traversal, link detection, text-based magnet detection, magnet/.torrent URI patterns, MutationObserver, iframe, Shadow DOM, site-specific patterns, performance, complete impl, edge cases, test cases.

Timing (Dim08 §1): `document_idle` default & recommended; for heavy SPAs (Gmail-class) wrap in `window.onload` readyState checks (`safeInitialize`). `initializeWhenReady` handles loading/interactive/complete.

Traversal (Dim08 §2): Fast path = `querySelectorAll('a[href^="magnet:"]')` / `a[href$=".torrent"]` / `[data-magnet]`. **TreeWalker** (NodeFilter.SHOW_TEXT) for text-node magnet scanning, skipping script/style/noscript (per MDN faster than recursive walking). Recursive traversal needed for Shadow DOM (with visited Set + maxDepth 100).

Detection (Dim08 §3–6): `LinkScanner` scans anchors, onclick handlers, data attributes, per-site selectors. Regexes: `MAGNET_FULL = /magnet:\?xt=urn:btih:[a-fA-F0-9]{32,40}(?:&[^\s"'<>]+)*/gi`; `BTIH_EXTRACT = /xt=urn:btih:([a-fA-F0-9]{32,40})/i`; hash 32=base32, 40=hex. `.torrent` patterns: DIRECT `/\.torrent(?:\?.*)?$/i`, download endpoints (`/download.php?...torrent`, `/torrent/download/`, etc.), file hosts (torcache/itorrents/zoink/etc.). BitTorrent v3.1 (Tixati, medium confidence): `urn:btih-sha3:...`.

MutationObserver (Dim08 §7): Fires on microtask queue — **~88× faster than setTimeout polling**. Debounce **500ms**, maxBatchSize 100, filter childList added nodes; incremental scan only new nodes. IntersectionObserver (rootMargin 200px) for infinite scroll.

iframe (Dim08 §8): `all_frames:true` injects into matching frames; cross-origin iframe DOM inaccessible from JS — content script with all_frames handles it. `match_about_blank:true` runs in dynamically-created blank iframes. `window.self !== window.top` detects iframe context.

Shadow DOM (Dim08 §9): `querySelectorAll` does NOT pierce shadow boundaries — recurse `node.shadowRoot` manually (visited WeakSet); `createNodeIterator` to find shadow hosts; `salesforce/kagekiri` library option.

Site patterns (Dim08 §10): SITE_PATTERNS DB for thepiratebay (domains thepiratebay.org/.se/piratebay.live/baymirror.com; TPB title `Download this torrent using magnet`), 1337x (1337x.to/.st/x1337x.ws), nyaa (nyaa.si/.net), rutracker (rutracker.org/.net/.nl), generic fallback. `detectSitePattern()` by hostname.

Performance (Dim08 §11, summary): debounce 500ms, 16ms/frame budget (yield to browser), dedupe by infohash, DOM-first, chunking; cleanup (`ScannerCleanup` tracks observers/timers/listeners, disconnect on navigation — memory-leak prevention).

Edge cases (Dim08 §13): URL-encoded magnet hrefs, `javascript:` URLs containing magnets, base32 hashes, data-* attributes (incl. data-hash 40-char), dedupe normalize by hash, safe querySelectorAll, memory-leak cleanup.

Key technical decisions (Dim08 §14 summary): document_idle + readyState checks; CSS selectors + TreeWalker + MutationObserver combo; debounced 500ms incremental; 16ms frame budget; dedup by infohash; all_frames:true + match_about_blank:true; shadow DOM recursive. Browser support: MutationObserver Chrome26+/FF14+/Safari7+/Edge12+; IntersectionObserver Chrome51+; Shadow DOM v1 Chrome53+.

Cross-refs: Dim03 (content scripts), Dim06 (magnet detection), Dim10 (UI). Insight 3 uses Dim08's "90% of magnets via querySelectorAll" finding.

Dimension 09 — Extension ↔ Boba API Integration

Covers (Dim09 §1–16 + appendices): system architecture, qBittorrent API reference, **INFERRED** Boba endpoints, extension CORS privileges, auth strategies (cookie/JWT/apikey), complete API client, retry logic, SSE/WebSocket management, credential storage schema, health checks, offline queue, rate limiting/batching, config schema, manifest, security.

> **NOTE — port/endpoint discrepancy:** Dim09 was written earlier/inferentially and uses qBittorrent + Boba on **port 8080** and INFERS Boba endpoints (`/api/v1/health`, `/api/v1/torrents`, JWT auth with `/api/v1/auth/login|refresh`). Dim01 (actual repo analysis) has qBittorrent on **7185**, FastAPI on **7187**, no JWT, and real endpoints `/api/v1/download`, `/api/v1/search/stream/{id}`, `/`

> `health`. Cross_verification resolves C1 (auth: support BOTH cookie + JWT/apikey) and C4 (ports: both correct, user-configurable). **The implementation should treat Dim01 as authoritative for actual Boba endpoints and Dim09 as the client-architecture/resilience reference.**

Communication patterns (Dim09 §1.2): fetch (extension→server), EventSource (SSE search results), WebSocket (real-time progress), runtime.sendMessage (popup↔SW), storage.local (config/creds/queue), alarms (health checks/sync/keep-alive).

Auth strategies (Dim09 §5): Comparison — Cookie/SID (qBittorrent direct, needs `credentials:'include'`, CSRF), JWT Bearer (Boba Go backend, stateless, token expiry mgmt), API key (FastAPI search), Basic (legacy). `JWTAuth` with 60s-buffer refresh, single-flight refresh promise (prevents simultaneous refreshes), persisted to storage.local, **password NOT stored**.

API client (Dim09 §6): Adds torrents direct (qBittorrent multipart) with fallback to Boba API; getTorrents/getTransferInfo/syncMainData(rid)/deleteTorrent/pauseTorrent/resumeTorrent.

Retry (Dim09 §7): Error classification table — retryable: 0/AbortError, 429 (respect Retry-After), 500/502/503/504, 401 (refresh-then-retry); non-retryable: 400/403/404/415. **Exponential backoff with jitter** (DEFAULT_RETRY_CONFIG: maxRetries 3, baseDelay 1000ms, maxDelay 30000ms, backoffMultiplier 2, jitterFactor 0.5, retryableStatusCodes [0,429,500,502,503,504]). Request/response interceptor pattern for 401→refresh.

SSE/WebSocket (Dim09 §8): EventSource **cannot send custom headers** — auth via URL query param (`api_key=`). Reconnect with exponential backoff (1000·2^attempts, cap 30000ms). **Chrome 116+ keeps SW alive while WebSocket active.** WebSocketManager with ping (30s) + message queue. SW keep-alive: `chrome.alarms periodInMinutes:4` (< 5min SW kill).

Credential storage (Dim09 §9): Storage schema keys (boba_config, boba_sid, boba_jwt, boba_refresh_token, boba_credentials{username,rememberMe — NO password}, boba_api_key, boba_offline_queue, boba_search_history, boba_preferences, boba_cached_torrents, boba_categories, boba_tags). **chrome.storage.local NOT encrypted — store tokens only, never plaintext passwords; use chrome.identity for OAuth; token expiry+cleanup; clear on logout; HTTPS only; validate certs.**

Health checks (Dim09 §10): Separate liveness (`/api/v1/health`) from readiness (`/api/v1/health/ready`); parallel checks (boba/search/qbittorrent) with `AbortSignal.timeout(5000)`; status healthy/degraded/unhealthy.

Offline queue (Dim09 §11): Persist to storage.local, retry on reconnect, max 3 attempts per item, optimistic return when queued. `ConnectivityManager` on online/offline events (`navigator.onLine`). [Insight 6 strengthens this: navigator.onLine unreliable for LAN — use proactive `/app/version` ping every 30s via chrome.alarms.]

Rate limiting (Dim09 §12): Token bucket (strongest default). Per-endpoint limits: search {rps:2,burst:3}, torrents/add {5,10}, torrents/info {10,20}, sync/maindata {2,5}, transfer/info {5,10}, default {10,20}. RequestBatcher (maxBatchSize 10, maxWaitMs 50).

Config defaults (Dim09 §13.2): bobaBaseUrl `http://localhost:8080`, searchApiUrl `http://localhost:7187`, qbittorrentUrl `http://localhost:8080`, authType `cookie`, requestTimeout 30000, maxRetries 3, retryBaseDelay 1000, retryMaxDelay 30000, rateLimitRps 10, rateLimitBurst 20, enableOfflineQueue true, maxQueueSize 100, healthCheckInterval 30000, healthCheckTimeout 5000, sseReconnectDelay 5000, searchTimeout 30, searchLimit 100, syncInterval 2000, backgroundSync true.

Manifest (Dim09 §14): permissions storage/alarms/notifications/contextMenus/activeTab; host_permissions localhost+127.0.0.1 (http+https, * port); `externally_connectable` matches localhost. **Do NOT use `<all_urls>` in production — specify exact Boba URLs.**

Cross-refs: Dim01, Dim02, Dim11.

Dimension 10 — Extension UI/UX Design Patterns

Covers (Dim10 §1–12 + appendices): popup, options page, context menus, action/badge, notifications, keyboard shortcuts, side panel, icon states, dark/light theme, responsive design, accessibility, i18n.

Popup (Dim10 §1): Chrome hard limits **800×600 max / 25×25 min** (Chromium `kMaxSize={800,600}`); sweet spot **380–450px wide** (top extensions ~400×500); recommended 300–400px wide × 400–600px tall. Use CSS reset, explicit body width (not 100%), `overflow-y:auto`, system font stack. Boba popup uses `--popup-width:400px`, min 400/max 580px height, fixed header (48px) + connection bar (28px) + scrollable main + footer (36px). States: empty / torrent-list / sending (progress ring) / success.

Options page (Dim10 §2.4): `chrome.storage.sync` for settings (100KB total, 8KB/item), `storage.local` for larger data, `storage.session` for temp MV3 state; storage API preserves types (no JSON.parse needed); `onChanged` listener.

Context menus (Dim10 §3): require `contextMenus` permission; `targetUrlPatterns`; types normal/checkbox/radio/separator; >**1 visible item auto-collapses into a parent labeled with extension name**.

Badge/action (Dim10 §4): badge text **max ~4 chars visible**; color RGBA[0-255] or CSS string; per-tab via tabId (auto-resets on tab close); setIcon accepts path object or ImageData.

Notifications (Dim10 §5): 4 templates (basic/image/list/progress), up to 2 buttons; events onClicked/onButtonClicked/onClosed/ onPermissionLevelChanged. **macOS Chrome 59+ shows native notifications (no images, list shows only first item).**

Keyboard shortcuts (Dim10 §6): `commands` — **max 4 suggested keys/ extension**, all must include Ctrl or Alt; `_execute_action` (MV3); fires `chrome.commands.onCommand` in SW. Mac: 'Ctrl'→'Command' by default, use 'MacCtrl' for actual Control. Boba commands: send-all-torrents (Ctrl+Shift+B), scan-page (Ctrl+Shift+S), open-dashboard (Ctrl+Shift+D), toggle-side-panel (Ctrl+Shift+P), _execute_action (Ctrl+Shift+U).

Side panel (Dim10 §7): Chrome 114+, MV3, `sidePanel` permission; opened by action click (setPanelBehavior), user gesture (sidePanel.open — needs tabId/windowId), context menu; **cannot auto-open programmatically without user interaction**; sidePanel.open() Chrome 116+, close() Chrome 141+. [Cross_verification #30 (low confidence): side panel Chrome 114+ limits browser support.]

Icons (Dim10 §8): sizes 16 (favicon/menu), 32 (Windows high-DPI), 48 (CWS), 128 (CWS listing); action toolbar uses 16/24/32. Recommended set includes state variants: default, detecting, sending, error, offline, success.

Theme (Dim10 §9): dark via `prefers-color-scheme` media query + `<meta color-scheme>`; CSS variables; JS detect via `matchMedia`.

Responsive/accessibility/i18n (Dim10 §10–12): popup max 800×600 / recommended 380–450px / content-driven height with overflow-y:auto. ARIA: aria-label for icon-only buttons, role for custom widgets, visible focus indicators. i18n: `_locales/<locale>/messages.json`, manifest MUST define `default_locale` if _locales exists, `__MSG_key__` in manifest/ CSS, `chrome.i18n.getMessage()` in JS, up to 9 placeholders; predefined @@ui_locale/@@bidi_* for RTL.

API version reference (Dim10 C): chrome.action (Chrome 88), storage.session (102), sidePanel (114), sidePanel.open (116), sidePanel.close (141), notifications (28), contextMenus (6), commands (35), i18n (1), storage.sync/local (1).

Cross-refs: Dim01 ("send to" satellite UI — Insight 5), Dim09 (badge sync — Insight 9), Dim05 (tab group UI).

Dimension 11 — Security Model & Privacy Architecture

Covers (Dim11 §1–18): STRIDE threat model, manifest permissions, host pattern matching, credential encryption, content-script isolation, CSP, HTTPS enforcement, certificate validation for self-hosted Boba, input validation, rate limiting, privacy architecture, secure communication, update security, sandboxed iframes, a security checklist, privacy policy template.

Design principles (Dim11 §1): Least Privilege, Defense in Depth, Zero Trust, Secure by Default, Privacy First.

STRIDE (Dim11 §2): Spoofing (S1–S4: cert pinning/HTTPS, origin validation, isolated world/strict matches, magnet hash validation), Tampering (T1–T4: store-signed packages, HTTPS/HSTS, AES-GCM encryption, infohash validation), Repudiation (R1–R2: optional audit log), Information Disclosure (I1–I5: AES-256-GCM, minimal matches no `<all_urls>`, restrict web_accessible_resources + use_dynamic_url, sender validation, no telemetry of server URLs), DoS (D1–D3: client-side rate limiting, document_idle, storage quotas), Elevation of Privilege (E1–E3: sender validation + action allow-listing, keep Chrome updated, no eval + strict CSP). Trust boundaries TB1–TB4.

Permissions (Dim11 §3): Minimal manifest permissions = storage, alarms, notifications, activeTab; optional clipboardWrite; host_permissions = localhost only (http+https);

optional_host_permissions `https://*/*`. **MV3 CSP is an object** (`extension_pages` + `sandbox`) — `script-src`/`object-src`/`worker-src` only `'self'`/`'none'`/`'wasm-unsafe-eval'`; no unsafe-inline/unsafe-eval/remote scripts. **NEVER request** `<all_urls>`, `tabs`, `cookies`, `webRequest`, `scripting` (if declared in manifest), `downloads`, `history`.

Host patterns (Dim11 §4): Use specific per-site `matches` (thepiratebay/1337x/nyaa/rutracker/yts/eztv/…). **Wildcard `matches` nullify postMessage event-source protection** (any origin can trigger the channel) — avoid wildcards.

Credential encryption (Dim11 §5) — concrete parameters: AES-256-GCM + **PBKDF2-HMAC-SHA256, 100,000 iterations**; salt 128-bit (16 bytes) random per encryption; IV 96-bit (12 bytes) random per encryption; non-extractable CryptoKey (not persisted); master password never stored (provided per session); only encrypted data on disk. Decrypted creds cached in `chrome.storage.session` (memory-only, lost on restart). `chrome.storage.local` is NOT encrypted by default.

Content-script isolation (Dim11 §6): isolated world — no access to page JS variables/functions, shares only DOM; prevents page JS reaching extension APIs.

CSP (Dim11 §7): Default extension_pages `script-src 'self'; object-src 'self';`. Boba CSP: `default-src 'self'; script-src 'self'; object-src 'self'; connect-src 'self' https:; img-src 'self' data:; style-src 'self' 'unsafe-inline';`.

HTTPS / certificates (Dim11 §8–9, §13): HTTPS-only (except localhost); HSTS. Self-hosted Boba self-signed cert: certificate bundling/pinning OR instruct user to install self-signed CA into system trusted root; optional fingerprint verification. Chrome respects system cert store.

Input validation (Dim11 §10): v1 magnet = `xt=urn:btih:` 40 hex (or 32 base32); v2 = `urn:btmh:` 64 hex; validate info-hash length; sanitize display names (textContent not innerHTML); validate Boba URL (protocol/hostname/no embedded credentials); **reject `javascript:`/`data:`/`vbscript:` protocols; block dangerous ports.**

Rate limiting (Dim11 §11): `chrome.alarms` (MV3-recommended, survives SW termination + browser restart). Presets: conservative {10 req/60s}, normal {30/60s}, relaxed {60/60s}, custom. User-configurable.

Privacy (Dim11 §12): Data matrix — collected: Boba URL/creds (encrypted local, until deleted), magnet links/torrent names/tab URLs (ephemeral, immediate discard); NOT collected: page content, browsing history, IP. Analytics/error-reporting **opt-in only** (analytics 30-day, errors 7-day local). Chrome Web Store "Limited Use" policy. Defaults: enableAnalytics false, enableErrorReporting false, autoClearHistory true, historyRetentionDays 7, requirePasswordOnStartup true, autoLockTimeoutMinutes 30, verifyCertificates true, enforceHttps true. `eraseAllData()` for GDPR right-to-erasure. **No third-party analytics; no data selling.**

Secure communication (Dim11 §13): Set `credentials:'omit'` and `referrerPolicy:'no-referrer'` on all fetch.

Update security (Dim11 §14): Distribute only through official stores (cryptographically signed); Firefox self-hosted update manifest on HTTPS with `update_hash`; verify permissions didn't change on update + notify users.

Sandboxed iframes (Dim11 §15): for untrusted content rendering when needed.

Security checklist (Dim11 §16) — consolidated mandatory items: min permissions; host_permissions separate; optional_permissions for non-essential; never <all_urls>/tabs/cookies/history/downloads; `run_at:document_idle`; `all_frames:false` unless required; `use_dynamic_url:true`; restrict web_accessible_resources to matches; never include HTML in web_accessible_resources; strict CSP; AES-256-GCM creds; PBKDF2 ≥100k iterations; never store master password; storage.session for decrypted cache; clear on restart; "Clear All Data" function; validate `sender.id` + `sender.url` (chrome-extension://) on ALL onMessage handlers; action allow-listing; HTTPS for external; block non-HTTPS Boba (except localhost); `credentials:'omit'` + `referrerPolicy:'no-referrer'`; validate magnet links (regex + info-hash length 40 v1/64 v2); sanitize display names (textContent); reject javascript:/data:/vbscript:; block dangerous ports; client-side rate limiting via alarms; user-configurable limits; official-store distribution; HTTPS update manifests; no eval/new Function/inline scripts; no innerHTML with untrusted data; no hardcoded keys/creds/URLs; no remote script loading; no analytics without opt-in; privacy policy.

Cross-refs: Dim03 (isolated world, cross_verification #12), Dim04 (cross-browser permissions), Dim09 (auth/storage). Insight 7 (privacy-first activeTab) is medium confidence — some reviewers prefer declared host_permissions.

Dimension 12 — Testing, Build System & Store Distribution

Covers (Dim12 §1–19 + appendices): tech stack, Jest setup, browser-API mocking, Playwright E2E, content-script/SW testing, build-tool selection, TypeScript config, lint/format, CI/CD, packaging, store submission, version management, coverage, cross-browser build, pre-commit hooks, dev-workflow README.

Key decisions (Dim12 §1): Build = **WXT** (Vite); test = **Jest (unit)** + **Playwright (E2E)**; mock = manual Jest mocks + **sinon-chrome**; CI = **GitHub Actions** matrix builds; versioning = **release-please** + **Conventional Commits**; stores = Chrome/Firefox/Edge/Opera/Yandex.

Tech stack versions (Dim12 §2): WXT ^0.20.x, Vite ^6.x, TypeScript ^5.7.x, chrome-types ^0.1.x, webextension-polyfill ^0.12.x, Jest ^29.x, Playwright ^1.49.x, ESLint ^9.x, typescript-eslint ^8.x, Prettier ^3.4.x, Husky ^9.x, lint-staged ^15.x, release-please ^16.x, web-ext ^8.x.

Testing (Dim12 §3–7): Jest preset ts-jest, testEnvironment jsdom, setupFiles mock-extension-apis. **Playwright requires `chromium.launchPersistentContext` with `--load-extension` + `--disable-extensions-except`** (extensions attach to browser profiles at launch, not individual tabs — needs persistent context). Firefox E2E may have extension-ID extraction limits (cross_verification #29, low confidence). jsdom for DOM-scraping logic tests.

Build tool (Dim12 §8): WXT recommended (Vite under hood, Rollup production, sub-second dev start, sub-50ms HMR, supports all browsers + MV2/MV3, first-class TS, file-based entrypoints, automated publishing, 4K+ stars). Vite vs Webpack: <1s cold start vs 5–10s, <50ms HMR vs 1.5–3s. `npx wxt build -b chrome|firefox|edge|opera|safari`.

TS/lint (Dim12 §9–10): ESLint 9+ flat config (`eslint.config.mjs`); Prettier 3.4.x.

CI/CD (Dim12 §11): GitHub Actions main CI + release workflow + release-please. Build matrix browsers [chrome, firefox, edge] × node [18, 20, 22] (exclude edge on node 18), fail-fast false.

Packaging (Dim12 §12): WXT build + ZIP; manifest validation; CRX generation for self-distribution.

Store distribution (Dim12 §13) — concrete: - Requirements summary table: Chrome ($5 one-time, API/CI upload, <1h auto review, auto-publish yes, 2FA yes); Firefox (free, web-ext CLI, seconds auto + manual after, partial auto, 2FA yes); Edge (free, manual Partner Center, up to 7 days, no auto, 2FA yes); Opera (free, manual, manual review, no auto, no 2FA); Yandex (free, manual, unknown, no auto, no 2FA). - Chrome: $5 fee + OAuth 2.0 (CHROME_CLIENT_ID/SECRET/REFRESH_TOKEN/EXTENSION_ID), publish via `chrome-webstore-upload-cli`. - Firefox: API key (AMO_JWT_ISSUER/SECRET), `web-ext sign --channel=listed`; must set `browser_specific_settings.gecko.id`; submit source if minified. - Edge: Partner Center manual first version. - Opera: manual upload, manual review no SLA, accepts standard Chromium .zip. - Pre-submission checklist: manifest updated; icons 16/32/48/128/512; clear description; screenshots 640×480 or 1280×800; privacy policy URL; no remote code; CSP set; **no console.log in production**; source maps removed.

Versioning (Dim12 §14): Conventional Commits → SemVer (fix:→PATCH, feat:→MINOR, BREAKING CHANGE:→MAJOR). release-please creates Release PRs.

Coverage (Dim12 §15): coverageThreshold global 70% (branches/functions/lines/statements) [note: Insight 10 aspirationally targets 80%+]; reporters text/text-summary/lcov/html/json-summary; Codecov/Coveralls upload; optional SonarQube. [Insight 10: match Boba's enterprise bar — pytest/Playwright/SonarQube/Snyk/Semgrep/Trivy/Gitleaks; security scanning in CI; signed releases; CI auto-publishes to all 5 stores.]

Cross-browser (Dim12 §16): WXT auto builds; browser-specific code via `getBrowser()` (gecko in manifest → firefox, Edg/ → edge, OPR/ → opera); webextension-polyfill; build matrix in CI.

Pre-commit (Dim12 §17): Husky 9+ (`.husky/` + core.hooksPath) + lint-staged (staged files only).

Cross-refs: Dim01 (Boba quality bar), Dim04 (build/distribution), Dim11 (security scanning).

Insights (boba_extension_insight.md) — all 10

1. **Zero-Config Discovery** (Dim01+09+04, High): Auto-discover services by trying well-known localhost ports (7187, 7189, 8080) + `/app/version` / `/api/v2/app/version` health pings → one-click "Auto-Discover" vs 5+ manual fields. Mirrors Sonarr/Radarr.
2. **Unified Torrent Identity** (Dim06+07+09, High): Normalize every discovered item to `{infoHash, name, source, type:'magnet'|'torrent-file'|'torrent-url'}` — magnet (`xt=urn:btih`) and .torrent (SHA-1 of bencoded info) converge on the same 40-char BTIH. Enables dedup across a whole tab group + "already in queue" indicators; aligns with Boba's own infohash dedup.
3. **Progressive Enhancement Content Strategy** (Dim08+06+10, High): 3-tier detection — (1) universal magnet/.torrent link scan (catches ~90% via `querySelectorAll('a[href^="magnet:"]')`), (2) site-specific selectors for top ~20 sites, (3) text-based magnet detection for forums/Reddit + MutationObserver. Works on ANY site immediately (unique vs competitors).
4. **Yandex Tab Group as Batch Job** (Dim05+01+10, High): Right-click group → "Send All to Boba" → extract all tab URLs → parse each → submit as batch to `/api/v1/download` (multiple URLs newline-separated). Differentiator — no competitor treats tab groups as batch operations.
5. **Extension as Boba Satellite** (Dim01+09+11, High): Design as a satellite ("send to") client, not a management tool — use Boba's auth model (API keys, not its own), respect Boba config (categories/save paths), extend Boba UI not duplicate it. ~30% less code, single config source.
6. **Offline-Aware Queue** (Dim09+03+11, High): Boba is self-hosted → frequently offline. Detect offline → queue in `chrome.storage.local` → retry exponential backoff → sync when back. **navigator.onLine unreliable for LAN** — proactive `/app/version` ping every 30s via chrome.alarms + persistent queue.
7. **Privacy-First Permission Model** (Dim11+03+04, **Medium** — some store reviewers prefer declared host_permissions): Use `activeTab` primary + optional host permissions for popular sites; inject content scripts via `chrome.scripting.executeScript()` on demand. Least privilege, better store approval. Tradeoff: requires a click before detection.
8. **Multi-Client Gateway** (Dim02+09+01, **Medium** — adds complexity, v2 feature): Abstract a `TorrentClient` interface with pluggable

adapters (qBittorrent native, Transmission RPC, Deluge WebUI, rTorrent). Future-proof; Boba is one adapter.

9. **Real-Time Badge Sync** (Dim03+09+10, High): Poll `/sync/maindata` with incremental RID → badge text = download count, color = status (green downloading, blue complete, red error) via `setBadgeText`/`setBadgeBackgroundColor`. Ambient awareness, no notification spam.
10. **Enterprise-Grade Testing** (Dim12+01+11, High): Match Boba's bar — **80%+ coverage**, automated E2E with real Boba in Docker, security scanning (Sonar/Snyk/Semgrep/Trivy/Gitleaks) in CI, signed releases, CI auto-publish to all 5 stores. Most extensions have zero tests — this is a differentiator.

Cross-Verification (boba_extension_cross_verification.md)

Date 2026-06-06; 12 dimensions; 27,560 research lines. Tiers: 14 High / 11 Medium / 5 Low / 4 Conflicts (all resolved). Stats: High 45%, Medium 35%, Low 16%.

High confidence (14): qBittorrent cookie auth via `/api/v2/auth/login` → SID; `/torrents/add` accepts magnet in `urls` + .torrent via multipart `torrents`; MV3 requires service workers (Chrome/Opera/Chromium); chrome.tabGroups needs Chrome 89+/MV3/tabGroups+tabs perms; magnet format `magnet:?xt=urn:btih:<40-hex>&dn&tr`; BTIH = SHA-1 of bencoded info dict; bencode types i/string/l/d; extensions bypass CORS for host_permissions via SW fetch; Boba FastAPI :7187 / Go :7189 / qBittorrent typically :8080; webextension-polyfill bridges browser.* across Chromium; Yandex Chromium v147+ supports chrome.* (browser://tune/); content scripts isolated world; MV3 storage.local OS-encrypted + survives restart; **WXT recommended build tool**.

Medium confidence (11): Boba uses JSON persistence (no relational DB) — in-memory dataclasses; qBittorrent v5.0 renamed pausedUP→stoppedUP / pausedDL→stoppedDL; **Boba has 48 plugins (40 public + 8 private)**; Offscreen Documents enable DOM ops SW cannot; Firefox MV3 uses Event Pages (not SW); Opera unique `sidebarAction`; Yandex own extension store (separate from CWS); TreeWalker faster than recursive DOM walk; **debounced MutationObserver at 500ms ~88× faster than polling**; **AES-256-GCM + PBKDF2 100k iterations recommended**; WXT auto cross-browser builds.

Low confidence (5, needs verification): Boba SSE at `/api/v1/search/stream/{id}` — path not confirmed in official docs; exact Boba endpoint paths may vary (Python + Go backends); **qBittorrent API key auth (Bearer) only v5.2.0+**; Firefox E2E Playwright may have extension-ID extraction limits; side panel API Chrome 114+ limits browser support.

Conflicts found + resolutions (4, ALL resolved): - **C1 — Auth method for Boba APIs:** Dim02 says cookie-based, Dim09 says JWT/API key. **RESOLVED: both valid — direct qBittorrent uses cookies; Boba's own APIs may use JWT. Extension should support both.** - **C2 — Build tool:** Dim12 (WXT) vs Dim03 (Vite). **RESOLVED: WXT is built on Vite — use WXT as the framework.** - **C3 — MV3 vs MV2 for Firefox:** Dim03 (MV3 SW) vs Dim04 (MV2 for compat). **RESOLVED: use MV3 everywhere; WXT handles Firefox Event Pages automatically.** - **C4 — Boba port numbers:** Dim01 mentions 7187 (FastAPI) + 7189 (Go); docker-compose may vary. **RESOLVED: both correct — 7187 primary Python API, 7189 Go backend; user-configurable in extension.**

Overall assessment: Findings "highly consistent" with strong authoritative sourcing; all 4 conflicts resolved; architecture well-understood and ready for implementation. 10 validated architectural decisions: (1) MV3 + SW for Chrome/Opera/Yandex/Chromium; (2) WXT build tool; (3) direct qBittorrent WebUI API integration; (4) Boba 48-tracker plugin backend; (5) Tab Groups fully supported in Yandex (Chromium 147+); (6) mature magnet parsing; (7) bencode parsing feasible in browser JS/TS; (8) CORS bypassed via host_permissions from SW — no proxy needed; (9) AES-256-GCM credential encryption meets best practices; (10) Jest + Playwright industry-standard testing stack.

CONSOLIDATED HARD REQUIREMENTS / DECISIONS / CONSTRAINTS / RISKS (the implementation backbone)

Architecture decisions (mandated by research)

- **MV3 everywhere**, service-worker background (Chrome/Edge/Opera/Yandex/Chromium); Firefox uses Event Pages (handled by WXT).
- **WXT** (Vite-based) as the cross-browser build framework; per-browser manifest generation.
- **webextension-polyfill** for `browser.*` Promise API across all targets; do own runtime feature detection for Firefox-only APIs.

- Extension = **satellite "send-to" client** of Boba (not a management tool); use Boba's config/auth, extend (not duplicate) Boba's UI.
- **All cross-origin fetch in the service worker** (content scripts cannot cross-origin fetch since Chrome 85); bypass CORS via `host_permissions`.
- **Auto-discovery** of services via well-known localhost ports + health endpoints; manual config fallback.
- **Support BOTH** cookie auth (direct qBittorrent) AND JWT/API-key (Boba's own APIs) — C1 resolution.
- Treat **Dim01 endpoints/ports as authoritative** (qBittorrent :7185, FastAPI :7187, Go :7189; `/api/v1/search`, `/api/v1/search/stream/{id}`, `/api/v1/download`, `/api/v1/download/file`, `/api/v1/magnet`, `/api/v1/auth/status`, `/api/v1/config`, `/health`); Dim09's :8080 + JWT endpoints are inferred/client-architecture reference. Ports user-configurable (C4).

Hard technical constraints

- Service worker terminates at **30s idle / 5min single request / 30s fetch response** — use `chrome.alarms` keep-alive (`periodInMinutes` 0.5 for 30s tasks, 4 for SW-alive < 5min kill); register ALL listeners synchronously at top level; `return true` for async sendResponse.
- Storage quotas: local ~10MB, sync ~100KB total/8KB item, session ~10MB (in-memory). Badge text ≤4 chars.
- Popup max 800×600 / min 25×25; use 380–450px wide.
- EventSource cannot send custom headers — auth via URL query param.
- qBittorrent: Referer/Origin MUST match Host (CSRF); 403 = IP banned (use exponential backoff); 405 wrong method (v4.4.4+); 415 invalid torrent; 200 ≠ guaranteed added; search 409 if >5 concurrent; v5 state renames pausedUP→stoppedUP / pausedDL→stoppedDL.
- Infohash = SHA-1 of the **RAW bencoded info dict** (never decode-then-re-encode); Web Crypto SHA-1 needs HTTPS secure context.
- base32 (32-char) magnet hashes MUST be converted to hex (40-char).
- tabGroups: Chrome 89+/MV3; groupId not persistent across restarts (identify by title+color+URLs); collapsed groups still API-accessible; Firefox incompatible (Chrome/Yandex only); Chrome 148+ chrome://newtab cannot be grouped.
- Side panel Chrome 114+ only (open 116+, close 141+); cannot auto-open without user gesture.

- Firefox MV3 REQUIRES `browser_specific_settings.gecko.id`; version `x.y.z` only (no letters); data_collection_permissions required for new AMO submissions since Nov 2025.
- Opera does NOT support storage.sync/storage.managed; notifications partial on Mac.

Security/privacy requirements (mandatory)

- **Minimal permissions**: storage, alarms, notifications, activeTab; optional clipboardWrite; host_permissions localhost only (+ optional `https://*/*`). NEVER `<all_urls>`/tabs/cookies/webRequest/downloads/history.
- Specific per-site `matches` (never wildcard — wildcards nullify postMessage protection).
- **AES-256-GCM** + **PBKDF2-HMAC-SHA256 100,000 iterations**, 128-bit salt, 96-bit IV per encryption; non-extractable key; master password never stored; decrypted cache in storage.session only; "Clear All Data" (GDPR).
- Never store plaintext passwords; tokens only; clear on logout.
- Validate `sender.id` + `sender.url` (chrome-extension://) on ALL onMessage; action allow-listing.
- HTTPS only (except localhost); `credentials:'omit'` + `referrerPolicy:'no-referrer'` on fetch; cert validation/pinning for self-signed Boba.
- Validate magnet/info-hash (40 hex v1 / 64 hex v2); sanitize display names (textContent not innerHTML); reject javascript:/data:/vbscript:; block dangerous ports.
- MV3 strict CSP object (script-src/object-src 'self'); no eval/new Function/inline scripts/remote code/hardcoded creds.
- Client-side rate limiting via chrome.alarms (token bucket; per-endpoint limits; presets conservative/normal/relaxed); respect Retry-After.
- Analytics/error-reporting opt-in only; no third-party data sharing/selling; privacy policy; official-store distribution + signed updates.
- STRIDE-mapped mitigations across S/T/R/I/D/E.

Detection/parsing requirements

- 3-tier detection: universal link scan + site-specific selectors (top ~20 sites) + text-based + MutationObserver (debounce 500ms, ~88× faster than polling).
- DOM-first scanning; chunk large text (100KB) against catastrophic backtracking; handle iframes (all_frames:true + match_about_blank:true) + Shadow DOM (recursive shadowRoot).

- Unified identity normalization `{infoHash, name, source, type}`; dedup by infohash.
- Bencode parser must enforce sorted dict keys; reject invalid metainfo; size limit (≤10MB typical/50MB max); private-torrent passkey sanitization.

Resilience requirements

- Offline queue (storage.local, exponential backoff w/ jitter — base 1000ms, max 30000ms, mult 2, jitter 0.5, max 3 attempts); proactive health ping every 30s (navigator.onLine unreliable for LAN).
- Retry classification: retry 0/429/500/502/503/504/401-after-refresh; don't retry 400/403/404/415/parse errors.
- Real-time badge via `/sync/maindata` incremental RID polling (~2s); recommended sync 1–5s.

Testing/build/distribution requirements

- **Jest (unit) + Playwright (E2E with persistent context + –load-extension)**; jsdom for DOM tests; sinon-chrome mocking.
- Coverage threshold ≥70% (config) / aspirational 80%+ (Insight 10); reporters lcov/html/json-summary; Codecov/Coveralls; optional SonarQube + security scanners.
- GitHub Actions matrix (browsers × node 18/20/22); release-please + Conventional Commits → SemVer.
- Store assets: icons 16/32/48/128 (+512 for some); screenshots 640×480 or 1280×800; privacy policy; no console.log/source maps in prod; CSP set; no remote code.
- Per-store: Chrome $5 one-time + OAuth API publish + 2FA + MV3; Firefox API key + gecko.id + source-if-minified + data_collection_permissions; Edge manual Partner Center + 2FA; Opera manual review no SLA white-bg 612×408 screenshots; Yandex via CWS/browser://tune/.

Open questions / low-confidence items to verify before/during implementation

1. **Exact Boba endpoint paths** — Dim01 (`/api/v1/download`, `/api/v1/search/stream/{id}`) vs Dim09 inferred (`/api/v1/torrents`, `/api/v1/health`). SSE path `/api/v1/search/stream/{id}` is repo-derived but "not confirmed in official docs" (cross_verification #26–27). **Confirm against the live OpenAPI spec (`/openapi.json` on :7187) before coding the API client.**

2. **Boba's actual auth for its own APIs** — does the FastAPI service require any auth, or is it open (CORS *)? Dim01 shows `/api/v1/auth/qbittorrent` but no Boba-level JWT; Dim09 assumes JWT. C1 says support both — but the real default must be confirmed.
3. **Port defaults** — qBittorrent is :7185 (Dim01) not :8080 (Dim09 default). The extension's default config and auto-discovery list must include 7185/7186/7187/7188/7189/9117 (Dim01) plus 8080 fallback.
4. **qBittorrent API key auth (v5.2.0+ only)** — low confidence; only available on newer versions; cookie auth is the safe baseline.
5. **Firefox E2E with Playwright** — extension-ID extraction limitations (low confidence) — verify Firefox E2E approach.
6. **Privacy-first activeTab vs declared host_permissions** (Insight 7, medium) — some store reviewers prefer declared host_permissions; decide the permission model with store-approval risk in mind.
7. **Multi-client gateway** (Insight 8, medium) — explicitly deferred to v2; the v1 architecture should leave a `TorrentClient` seam but target Boba only.
8. **Boba's own `/api/v1/download` semantics** — does it accept magnet URIs and tracker URLs directly, or only `result_id` from a prior search? Dim01 DownloadRequest is `{result_id, download_urls}` — clarify whether arbitrary magnets (not from a Boba search) can be added directly, or whether the extension must add them straight to qBittorrent `/api/v2/torrents/add`.